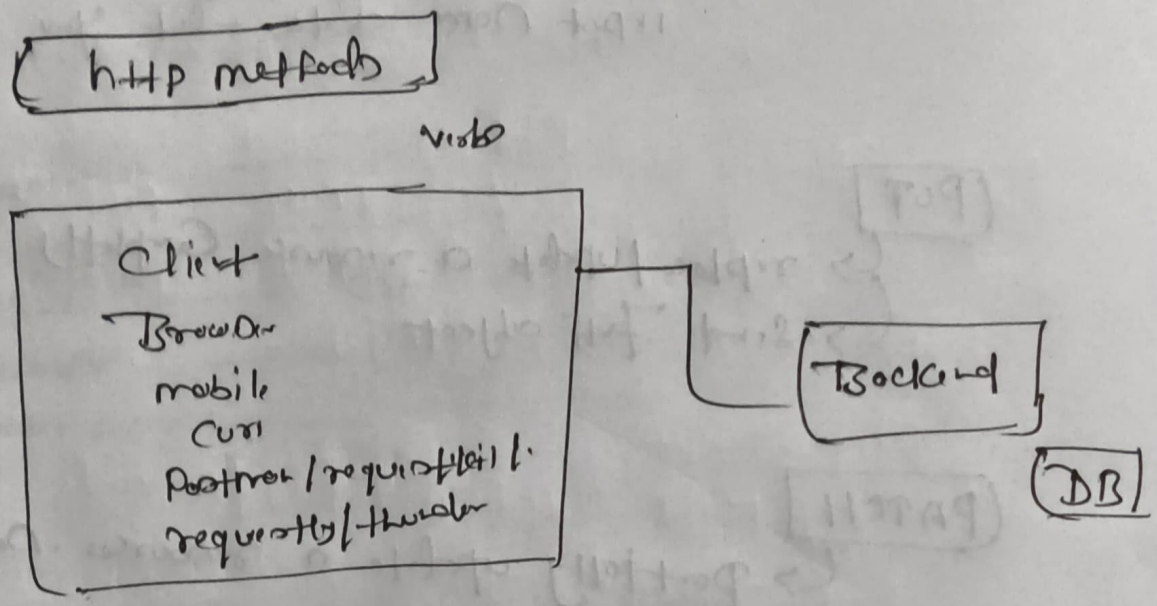
Backend Engineering Day-22

7 - March - 2026

Introduction to backend Engineering



GET

↳ To read / retrieve a resource

↳ no side effect

↳ (only read / no any change done)

(means one function not effect other function

↳ IDEMPOTENT (Calling 10 times = same as calling 1x)

It is a behaviour

you calling get method 10 times you always get same result. If you not change then you get same answer

↳ Cacheable by browser and CDN

↳ In GET, Nextjs usually do one prefetching means you just hover or link nextjs fromeworks background code that response. you feel it is too fast

POST
- → Create a new resources. NO IDEMPOTENT
- → Calling twice may create 10 resources
- → usually people thing post need to input some data but you leave it

PUT
- → replace / update a resource completely
- → send full object

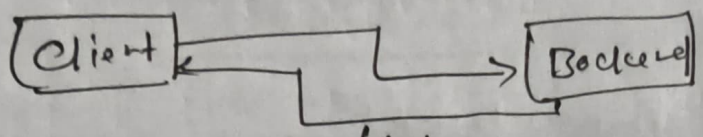
PATCH
- → Partially update a resources. Send only changed field

DELETE
- → Remove a Resources
- →

HEAD
- → Same as get but returns only ~~headers~~ HEADERS, no body

OPTIONS
- → Preflight / Capability Check
- → CORS

* Rarely used HEAD and OPTION



Status Code sent with response

201
Status Code

1. Information ——— (100 - 199)
2. Successful ——— (200 - 299)
3. Redirection ——— (300 - 399)
4. Client Error ——— (400 - 499)
5. Server error ——— (500 - 599

Chai.js
Console.log ("Chai Code is Running")

node --watch Chai.js
↳ Continously monitoring

node --version
↳ Check node version

npm --version
↳ Check npm version
↳ use someone Code in our Code / project

npm - owned Microsoft

⇒ npm init

package name

version : (1.1.0)

↳ Called semver versioning

↳ Semantic versioning

1.1.0

↳ PATCH → API changes

↳ MINOR → functionality change

↳ MAJOR →

description :

entry point : (index.js / app.js / server.js)

test command

keywords : chai, mocha, :

License :

type : "commonJS"

* npm init -y

↳ forcefully go to all

* <u>Package.Json</u>

<u>"main" : "index.js".</u>

TYPESCRIPT

→ Typescript is Javascript with syntax for types.

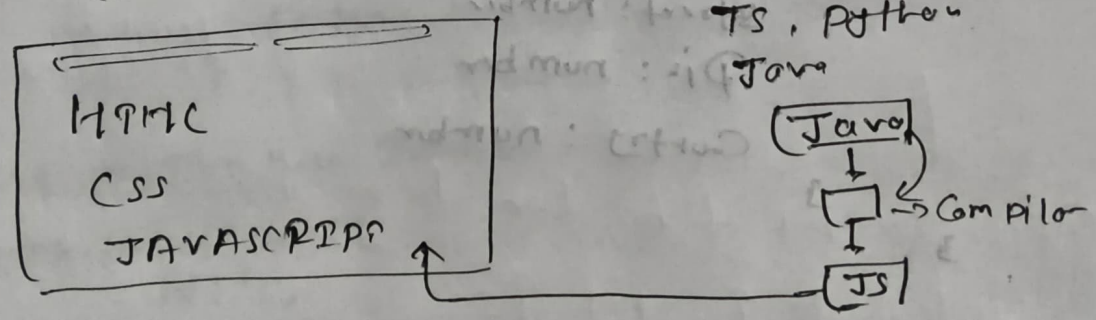
→ If you know JS, you know TS

```
function add (a, b) {
    return a+b;
}
console.log (add(1,2));//3
console.log (add ("piyush", true));// Piyushtrue
```

→ we not get Error, no JS is ~~loosely~~ loosely type Language / Interpreted

→ not give Error no it is design of JS

Browser



Browser not understand TS so we can add compiler which convert / compile in JS

→ Typescript never do Run time validation so we use like library Zod

my-script.id

```
// Ac In memory DB
// Save { 'User-1', { fname, lname } }

// Hashmap ( key, value )
//          String String

// { fname, lname, email, Contact : { mobile } , address {
        street, pin, country }

type UserID = String

interface User {
        id : UserID
        fname : String
        lname ? : String
        email : String
        Contact : {
            mobile : String
        }
        address : {
            street : number
            Pin : number
            Country : number
        }
    }

Class InMemoryDB {
        private _db: Map< User ID , User >

        Constructor () {

        }
```

```
Public insertUser (data : User) : UserID {
    if (this._db.has(data.id)) {
        throw new Error(`User with ID ${data.id}
            already exists`)
    }
    this._db.set(data.id, data)
    return data.id
}

Public updateUser(id: UserID, updateData : Omit<User, 'id'>
): Boolean {
    if (!this._db.has(id)) throw new Error(`User
    with ID ${id} does not exists`)
    this._db.set(id, {... updateData, id})
    return true
}
Public getUserByID (id: UserID): User {
    if (!this._db.has(id)) throw new Error(`User with
                id ${id} does not exists`)
    return this._db.get(id)!
}
}
```

```
Const myDb = new InMemoryDB()
myDb.insertUser({
    id: '1',
    fname: 'Piyush',
    email: 'piyush@email.com',
    Contact: {mobile: '9999'},
    address: {
        Country: 'In',
        Pin: 147001
        Street: 1
    }
})
```

```
myDb.updateUser('1', {
    fname: 'Piyush',
    email: 'piyush@email.com',
    Contact: {mobile: '9999'},
    address: {
        Country: 'In',
        Pin: 147001
        Street: 1
    }
})
```

my-ScriptJS

```
function add (a: number , b: number) {
    return a+b
}
add ( 3,
```

Note → JS में Suggestion काम नहीं देती

→ TS → Inside Suggestion work bcz

वो know the type of variable

→ TS give good developer Experience

```
function add (a: number , b: number ): string {
    return a.toString() + b.toString()
}
```

↓ function return type must be string

↓

→ This Code not to be Run because browser understand JS not TS

→ need to Compile

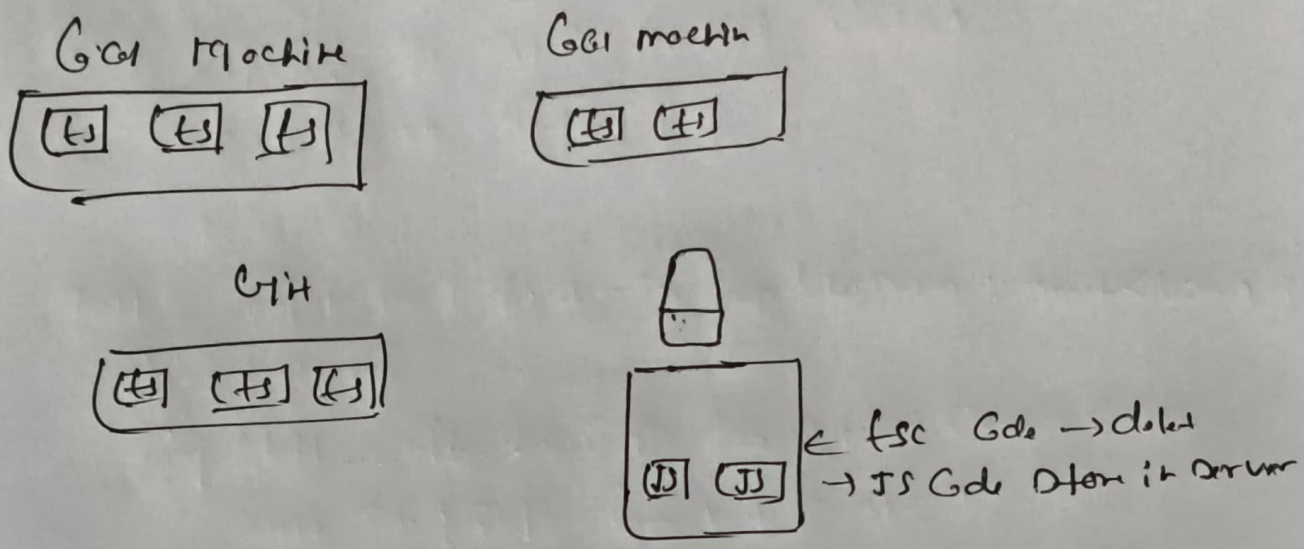
Sudo npm install typescript -g

↳ install in whole system

↳ not use -g that install on your current project

→ tsc --version

→ tsc my-script.js



→ TypeScript is a tool.

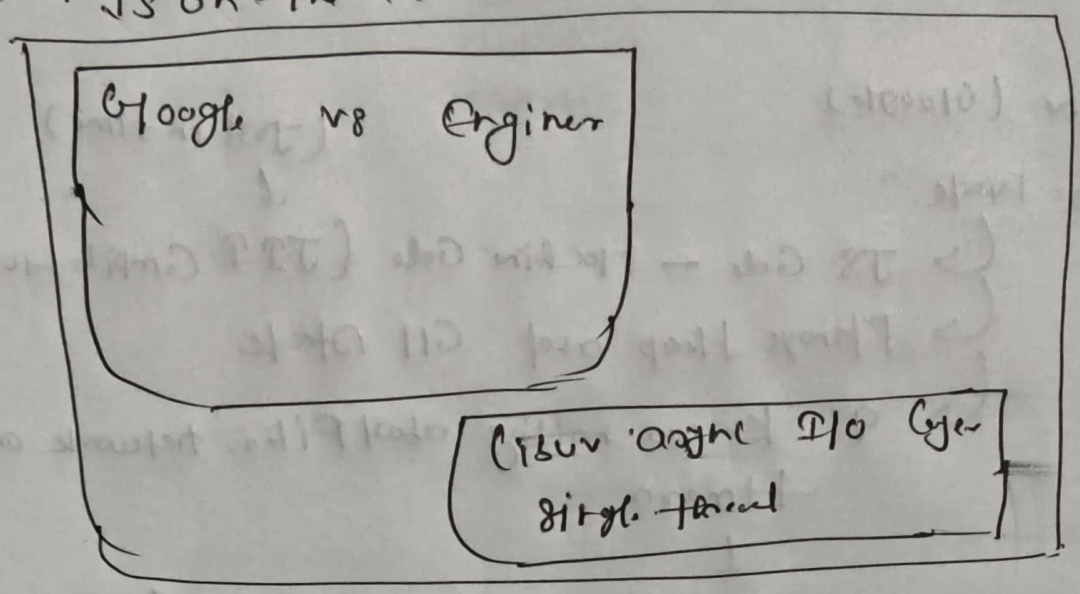
→ --save --dev
↳ Save as dev dependency

Class In

// HashMap (key, values)
// Interface name is Capital
// {fname, lname, email,
Contact: {mobile}
Address{Pin, Street, Country}

NodeJS

Not just JS on the Server



Methods to import Someone Code

```
Const fs = require("node:fs")
Const path = require("path")
Const os = require("os")

Console.log("NodeJS : ", process.versions.node)
Console.log("V8 " , process.versions.v8)
Console.log("libuv:", process.versions.uv);
```

```
//
NodeJS: 24.11.0
V8: 13.6.233.10-node.28
libuv: 1.51.0
```

```
Console.log("libuv:", process.platform);
```

↳ Darwin

Consoling ("OS", OS.Cpus().length); // 10
↳ 10 Cpus

V8 Engine (Google)
↳ Work
↳ JS Code — Machine Code (JIT Compilation)
(Just in time)
↳ Manage Heap and GC Stack
↳ It knows nothing about Files, Network or Timers
↓
V8 engine not know about these not NodeJS ☺

LIBUV (Pure C library)
↳ It provide you event Loop + thread Pool
↳ OS Level async I/O
↳ DNS Lookups

Node JS Bindings (C++)
(~~fs.readFile()~~)
↳ It work like bridge
↳ You write JS but in the back process done by libuv and I gave you back result

console.log (typeof global);

console.log (typeof globalThis);

⇓

object

object

Assignment

article one

① Node JS Architecture

↳ V8 engine

↳ libuv

↳ Node.js bridging

② global & globalThis

सोचो